Input Collector - Arquitectura y Documentación Técnica

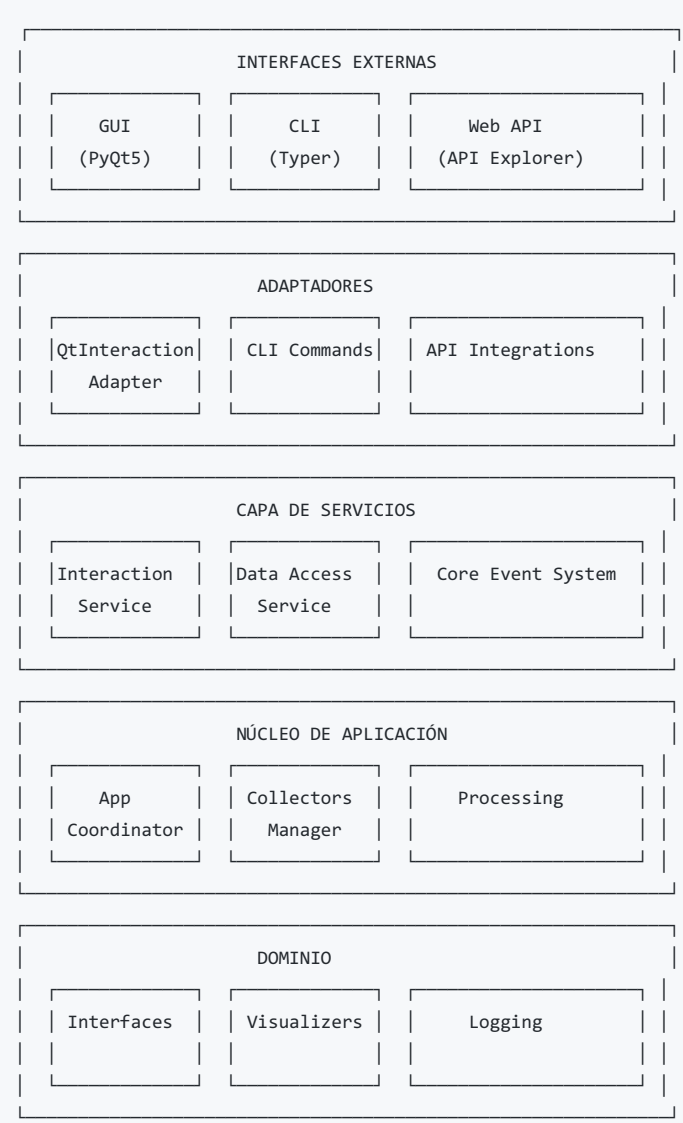
Resumen

Input Collector es una aplicación integral diseñada para capturar, procesar, visualizar y registrar eventos de entrada en tiempo real desde múltiples dispositivos (teclado, ratón, controladores PS4). La aplicación implementa una **arquitectura modular basada en eventos** que soporta múltiples interfaces de usuario (GUI, CLI, Web API) y proporciona capacidades extensibles para diferentes tipos de colectores y modos de visualización.

Arquitectura General

Patrón Arquitectónico Principal

La aplicación sigue una **arquitectura hexagonal (Ports and Adapters)** combinada con patrones **Mediator**, **Observer** y **Event-Driven Architecture**, organizando el código en capas bien definidas:



Submódulos y Componentes

1. Core Event System (`core/`)

Propósito: Sistema de eventos centralizado y framework-independiente para comunicación desacoplada entre componentes.

Componentes:

- **`EventDispatcher`**: Singleton thread-safe que gestiona suscripciones y dispatch de eventos
- **`EventType`**: Enumeración de tipos de eventos del sistema (INPUT, STATE_CHANGE, RECORDING_STARTED, etc.)
- **Funciones utilitarias**: API simplificada para uso común (`dispatch` , `subscribe` , `unsubscribe`)

Características:

- **Arquitectura dirigida por eventos**: Comunicación desacoplada entre módulos
- **Thread-safe**: Múltiples locks granulares para diferentes operaciones
- **Aislamiento de errores**: Los fallos en callbacks no afectan otros suscriptores
- **Sincronización tardía**: Almacenamiento del último evento para suscriptores posteriores
- **Framework-independiente**: No depende de Qt u otros frameworks

```
 # Ejemplo de uso mejorado
from core import dispatch, subscribe, EventType

def on_input_event(data):
    print(f"Received input: {data}")

def on_recording_started(data):
    print(f"Recording started: {data}")

# Suscripción a múltiples tipos de eventos
subscribe(EventType.INPUT, on_input_event)
subscribe(EventType.RECORDING_STARTED, on_recording_started)

# Dispatch con datos estructurados
dispatch(EventType.INPUT, {
    "key": "A",
    "timestamp": time.time(),
    "collector_type": "keyboard_mouse"
})
```

2. Interfaces (`interfaces/`)

Propósito: Definición de contratos y estructuras de datos fundamentales.

Componentes:

- `InputData`: Dataclass principal para eventos de entrada (25+ campos)
- `AbstractInputCollector`: Interfaz para colectores de entrada
- `AbstractVisualizer`: Interfaz para visualizadores
- `AbstractVisualizerProvider/Controller`: Protocolos para gestión de UI

Características:

- **Dataclass**: Soporte para todos los tipos de entrada y modos de tracking
- **Contratos consistentes**: Interfaces que garantizan comportamiento uniforme
- **Tipado estructural**: Protocolos Python para flexibilidad
- **Serialización**: Método `to_dict()` optimizado para logging y APIs

3. Collectors (`collectors/`)

Propósito: Captura de eventos de entrada desde diferentes dispositivos con arquitectura thread-safe y event-driven.

Componentes:

- `CollectorManager`: Gestor central con capabilities dinámicas
- `KeyboardMouseCollector`: Colector con múltiples modos de tracking
- `PS4ControllerCollector`: Arquitectura **event-driven** basada en hilos con SDL2 event loop

Características del PS4ControllerCollector:

- **Arquitectura completamente event-driven**: El colector procesa y despacha eventos en tiempo real a través de callbacks y suscripciones, desacoplando la captura de eventos del procesamiento y visualización.
- **Event loop dedicado**: SDL2 event loop en hilo dedicado para captura eficiente y asincrónica.
- **Reconexión automática**: Manejo de desconexiones
- **Event processing thread-safe**: Locks granulares y manejo de errores
- **Detección de cambios**: Tracking de estados previos para trigger/stick
- **Core event integration**: Dispatch automático a sistema de eventos

```
# Capacidades del PS4 Controller
{
    "supports_analog_sticks": True,
    "supports_triggers": True,
    "supports_haptic_feedback": False,
    "button_count": 14,
    "analog_stick_count": 2,
    "trigger_count": 2,
    "connection_type": "SDL2_GameController",
    "auto_reconnect": True,
    "thread_safe_events": True,
    "event_driven": True
}
```

Modos de Tracking de Ratón:

```
TRACKING_MODES = {
    "delta": "Raw movement deltas",
    "angle": "Direction and angle (8-directional)",
    "axis": "Cardinal directions (N/S/E/W)",
    "joystick": "Direction and speed (analog-like)"
}
```

4. Processing (`processing/`)

Propósito: Procesamiento, formateo y transformación de eventos con soporte extendido.

Componentes:

- `EventProcessor`: Procesador con soporte extendido para PS4

Características:

- **Formateo específico por colector**: Diferente handling para keyboard/mouse vs PS4
- **Detección de eventos especiales**: F9 para toggle recording, eventos de sistema
- **Formateo multi-modo**: Soporte para todos los modos de tracking de ratón
- **Procesamiento de ejes**: Handling especializado para joysticks y triggers PS4
- **Thread-safe**: Procesamiento seguro con señales Qt

5. Services (`services/`)

Propósito: Capa de servicios singleton mejorada para lógica de negocio.

Componentes:

- `InteractionService`: Servicio para control de aplicación expandido
- `DataAccessService`: Servicio para acceso a datos en tiempo real con categorización

Características:

- **Thread-safety mejorado**: Locks granulares para diferentes operaciones
- **Categorización de eventos**: Separación por tipo (key, mouse_movement, mouse_button, controller)
- **Sistema de callbacks**: Patrones reactivos para consumers externos
- **Gestión de memoria**: Límites de historial configurables
- **Estado metadata**: Tracking de información del colector activo

6. Visualizers (`visualizers/`)

Propósito: Componentes de visualización en tiempo real con interfaces avanzadas.

Componentes:

- `BaseVisualizer`: Clase base con capabilities support
- `KeyboardVisualizer`: Teclado virtual mejorado
- `MouseVisualizer`: Visualización joystick mejorada
- `MouseButtonsSelector`: Selector con edit mode
- `PS4ControllerVisualizer`: Interfaz de paneles divididos con logging en tiempo real

PS4ControllerVisualizer - Características:

- **Interfaz de paneles divididos**: Controller visual + panel de estado en tiempo real
- **Updates thread-safe**: Uso de señales Qt para actualizaciones desde hilos
- **Logging en tiempo real**: Panel de logs con colores y timestamps
- **Edit mode avanzado**: Snapshot/commit system para posiciones
- **Múltiples modos**: Image overlay + dibujado directo
- **Estado en vivo**: Display de valores de triggers, sticks y botones

```
# Arquitectura del panel dividido
┌─────────────────────────────────────────────┐
│                                             │
│  Controller Visualization                   │
│  (Image overlay + interactive elements)     │
│                                             │
└─────────────────────────────────────────────┘

┌─────────────────────────────────────────────┐
│ State Panel                                 │
│ ┌─────────────────┐ ┌─────────────────────┐ │
│ │ Active Buttons  │ │ Analog Values       │ │
│ │                 │ │ Left Stick: (0.2, -0.8)│ │
│ └─────────────────┘ │ Right Stick: (0.0, 0.0)│ │
│                     │ L2 Trigger: 0.0     │ │
│                     │ R2 Trigger: 0.0     │ │
│                     └─────────────────────┘ │
└─────────────────────────────────────────────┘

┌─────────────────────────────────────────────┐
│ Real-time Event Log                         │
│ [14:23:45] Button X pressed                 │
│ [14:23:45] Left stick moved to (0.2, -0.8)  │
│ [14:23:46] Button X released                │
└─────────────────────────────────────────────┘
```

7. Logging (`logging/`)

Propósito: Sistema de registro a múltiples formatos.

Componentes:

- `LoggingManager`: Coordinador central de logging
- `JSONLogger`: Logger para formato JSON estructurado
- `CSVLogger`: Logger para formato CSV tabular

Características:

- **Headers dinámicos CSV**: Generación automática basada en tracked keys
- **Formateo específico por colector**: Diferentes campos para keyboard/mouse vs PS4
- **Thread-safe signals**: Comunicación mediante señales Qt
- **Gestión automática**: Apertura/cierre automático de archivos

8. UI (`ui/`)

Propósito: Gestión de interfaz de usuario y coordinación de visualizadores con tema oscuro.

Componentes:

- `UIManager`: Gestor principal con tema oscuro
- `VisualizerManager`: Coordinador con capabilities awareness
- `KeyboardEventFilter`: Filtro para modo de edición

Características:

- **Tema oscuro**: Colores y estilos consistentes
- **Capabilities-aware**: UI adaptativa según el colector activo
- **PS4 integration**: Controles específicos para visualizador PS4
- **Edit mode coordination**: Gestión centralizada del modo de edición

9. App Coordinator (`app/`)

Propósito: Mediador central con integración al sistema de eventos core.

Componentes:

- `AppCoordinator` : Coordinador principal con core events

Características:

- **Core event integration**: Dispatch automático a sistema de eventos
- **Thread-safe coordination**: Uso de señales Qt para operaciones UI
- **Estado centralizado**: Gestión consistente de recording, edit mode, tracked keys
- **Error handling**: Manejo de errores en coordinación

10. Adapters (`adapters/`)

Propósito: Adaptadores thread-safe entre frameworks y servicios core.

Componentes:

- `QtInteractionAdapter` : Adaptador para operaciones UI thread-safe

Características:

- **Thread-safe execution**: Timeouts y error handling para operaciones cross-thread
- **Framework isolation**: Separación clara entre Qt y servicios core
- **API**: Support for all application operations

11. CLI (`cli/`)

Propósito: Interfaz de línea de comandos con capacidades de monitoreo avanzado.

Componentes:

- `CLIApp` : Aplicación principal CLI con Typer
- `CLIShell` : Shell interactivo estilo REPL
- `CLIConfig` : Gestión de configuración y estado compartido
- `HeadlessInputCollector` : Stack de aplicación para modo headless
- **Commands**: Módulos de comandos especializados
- `PS4ControllerMonitor` : Monitor dedicado para controladores PS4

Características del CLI:

Monitor Commands:

- **PS4ControllerMonitor**: Monitor dedicado para controladores PS4
- **Event-driven architecture**: Uso del core event system
- **Rich console interface**: Tablas y paneles en tiempo real
- **Cross-platform key detection**: ESC/Q para salir en Windows/Unix
- **Smart collector detection**: Monitoreo específico según colector activo

```
 # Monitoreo PS4 con display en tiempo real
$ input-collector monitor controller
┌─────────────────────────────────────────────────┐
│                PS4 Controller                   │
├─────────────────────────────────────────────────┤
│ Buttons: X, Circle                              │
│ Left Stick: (0.2, -0.8) → SE                    │
│ Right Stick: (0.0, 0.0) → Center                │
│ L2 Trigger: 128/255 (50%)                       │
│ R2 Trigger: 0/255 (0%)                          │
└─────────────────────────────────────────────────┘

Recent Events:
[14:23:45] Button X pressed
[14:23:45] Left stick moved SE
[14:23:46] L2 trigger: 50%
[14:23:46] Button X released

Press ESC or Q to exit...
```

Command Structure:

```
# Data access commands
input-collector data active-keys
input-collector data mouse-position
input-collector data controller-connected

# Interaction commands
input-collector interaction recording start
input-collector interaction collectors switch ps4_controller
input-collector interaction mouse set-mode joystick

# Advanced monitoring
input-collector monitor events --collector ps4_controller
input-collector monitor controller  # Auto-detects PS4
```

12. API Integration (archivos raíz)

Propósito: Exposición de APIs via web para acceso remoto con integración a eventos core.

Componentes:

- `InputCollectorAPI` : API estática para control de aplicación
- `InputDataAPI` : API estática para acceso a datos
- `InputCollectorApiIntegration` : Integración web para datos con core events
- `InteractionApiIntegration` : Integración web para control con core events

Características:

- **Core event integration**: WebSocket broadcasts automáticos via event system
- **PS4 state tracking**: Estado especializado para controladores PS4
- **Enhanced fallbacks**: Operación sin web server mejorada
- **Thread-safe operations**: Todas las operaciones API son thread-safe

Patrones Arquitectónicos Implementados

1. Event-Driven Architecture

- **Core Event System** como backbone de comunicación
- **Pub/Sub pattern** para desacoplamiento
- **Framework-independent** para flexibilidad

2. Patrón Mediator

- `AppCoordinator` actúa como mediador central
- **Core event integration** para comunicación externa
- **Thread-safe coordination** via Qt signals

3. Patrón Singleton

- **Thread-safe implementation** con double-checked locking
- **Granular locking** para diferentes operaciones
- **Error isolation** para estabilidad

4. Patrón Observer

- **Core event system** como implementación principal
- **Qt signals/slots** para UI thread-safety
- **Callback systems** en servicios

5. Patrón Strategy

- **Collector capabilities** dinámicas
- **Adaptive UI** según colector activo
- **Multiple tracking modes** intercambiables

6. Patrón Adapter

- **Framework isolation** mejorado
- **Thread-safe adaptations** con timeouts
- **Cross-platform compatibility**

Características Clave del Sistema

1. Arquitectura Event-Driven Completa

- **Sistema de eventos core** framework-independiente
- **Thread-safe communication** entre todos los componentes
- **Automatic event broadcasting** para APIs y CLI
- **Late subscriber synchronization** para estabilidad

2. Thread Safety

- **Multiple granular locks** en servicios y collectors
- **Qt signal/slot threading** para UI updates
- **SDL2 event loop threading** para PS4 controller
- **Error isolation** en callbacks y event handlers

3. Multi-Interfaz Avanzada

- **GUI**: Aplicación PyQt5 con split panels y real-time updates
- **CLI**: Shell interactivo con monitoreo específico por colector
- **Web API**: Control remoto con WebSocket real-time events

4. PS4 Controller Support

- **SDL2 integration** con thread-safe event processing
- **Automatic reconnection** para dispositivos inalámbricos
- **Advanced visualization** con image overlays
- **Real-time monitoring** en CLI con Rich console

5. User Experience Mejorada

- **Dark theme** consistente
- **Split panel interfaces** para información detallada
- **Real-time state displays** con live value updates
- **Cross-platform CLI** con key detection inteligente

6. Configurabilidad Avanzada

- **Dynamic capabilities** adaptación según dispositivo
- **Multiple mouse tracking modes** con parámetros ajustables
- **Edit mode** para personalización visual
- **Flexible key/button tracking** configuration

7. Logging y Monitoreo

- **Multi-format simultaneous logging** (CSV + JSON)
- **Dynamic CSV headers** basados en configuración
- **Real-time event monitoring** en CLI y GUI
- **WebSocket event streaming** para integración externa

8. Extensibilidad y Mantenibilidad

- **Clear module boundaries** con interfaces bien definidas
- **Event-driven communication** para loose coupling
- **Framework-independent core** para portabilidad
- **Error handling y recovery

Flujo de Datos Principal

```
1. Input Device → Collector (thread-safe) → InputData
2. InputData → Core Event System → Multiple Subscribers
3. InputData → EventProcessor → ProcessingResult
4. ProcessingResult → AppCoordinator → UI + Logging
5. Core Events → {API WebSocket, CLI Monitor, External Systems}
6. Thread-safe Updates → {GUI Panels, CLI Display, Web Clients}
```

Casos de Uso Principales

1. Desarrollo de Juegos

- **PS4 controller testing** con visualización en tiempo real
- **Input lag analysis** con timestamps precisos
- **Analog input mapping** para stick y trigger calibration
- **Multi-input coordination** testing

2. Investigación UX/UI

- **Mouse movement pattern analysis** con múltiples tracking modes
- **Keyboard usage heatmaps** con visualización en tiempo real
- **Accessibility testing** con monitoreo de patrones de uso
- **Cross-device interaction studies**

3. Automatización y Testing

- **Macro recording** con soporte multi-dispositivo
- **Automated UI testing** con event playback
- **Input validation** para aplicaciones
- **Performance benchmarking** de input handling

4. Monitoreo de Sistema en Tiempo Real

- **Live input monitoring** via CLI o Web API
- **Device connectivity tracking** con reconexión automática
- **Usage analytics** con logging estructurado
- **Remote monitoring** via WebSocket APIs

5. Desarrollo y Debugging

- **Event inspection** en tiempo real
- **Input device debugging** con estado detallado
- **API integration testing** with endpoints
- **Cross-platform compatibility verification**

Tecnologías y Dependencias

Core

- **Python 3.8+**: Lenguaje principal con typing moderno
- **PyQt5**: Framework GUI con thread-safe signals
- **dataclasses**: Estructuras de datos optimizadas
- **threading**: Concurrencia y synchronization

CLI Avanzado

- **Typer**: Framework CLI moderno con auto-completion
- **Rich**: Output avanzado con tablas, paneles y colors
- **Cross-platform**: Windows/Unix/macOS support

Input Capture Mejorado

- **pynput**: Captura keyboard/mouse cross-platform
- **pysdl2** (opcional): SDL2 binding para PS4 controller
- **Platform-specific APIs**: Integración con APIs nativas del SO

Web API y Comunicación

- **API Explorer**: Framework web para APIs documentadas
- **WebSocket**: Comunicación bidireccional en tiempo real
- **HTTP REST**: APIs stateless para operaciones

Visualización Avanzada

- **Qt Graphics**: Rendering optimizado para visualizadores
- **Image processing**: Overlay support para controller visualization
- **Real-time updates**: 60 FPS capable rendering

Arquitectura de Deployment

```
Input Collector Application
├── Standalone GUI Mode
│   ├── Full feature set
│   ├── Real-time visualization
│   └── Local file logging
├── CLI Headless Mode
│   ├── Minimal resource usage
│   ├── Scriptable operations
│   └── Remote monitoring
├── Hybrid GUI + CLI Mode
│   ├── Concurrent operation
│   ├── Shared state management
│   └── Coordinated cleanup
└── Web API Integration
    ├── Remote control capability
    ├── Real-time event streaming
    └── Cross-platform accessibility
```